Práctica 2
Listas doblemente enlazadas circulares

Samuel Carrasco Santurde
Alexander Tomás Echevarria Pizarro
Erasmo Seebold Guadilla

Índice general

1. Introducción **4**
1.1. Objetivos y alcance 4

2. Diseño de las clases **5**

3. Descripción de las estructuras de datos principales **7**

4. Diseño e implementación de los métodos principales **8**
4.1. isEmpty 8
4.2. size 8
4.3. isAlone 8
4.4. locate 8
4.5. removeFirst 9
4.6. removeLast 9
4.7. remove 10
4.8. first 10
4.9. last 10
4.10. contains 11
4.11. find 11
4.12. addWhenEmpty 11
4.13. insertAdd 12
4.14. actualizarFirst 12
4.15. insertFirst 12
4.16. iterator 12
4.17. toString 13
4.18. Resumen de costes comunes 13
4.19. addTo 14
4.20. UnorderedfindOrAdd 14
4.21. addToFront 14
4.22. addToRear 14
4.23. addAfter 15
4.24. Resumen de costes de la lista no ordenada 15
4.25. add 16
4.26. OrderedfindOrAdd 17
4.27. merge 18
4.28. combinarIguales 19
4.29. Resumen de costes de la lista ordenada 19
4.30. Pruebas realizadas 19

5. Código **21**

6. Conclusiones **22**
6.1. Dificultades 22
6.2. Tiempos y comparaciones del merge elegido 23
6.3. Tiempos y comparaciones de otros merges 24

Índice de tablas

4.1. Costes de las operaciones comunes. 13
4.2. Costes de las operaciones de la lista no ordenada. 15
4.3. Costes de las operaciones de la lista ordenada. 19

6.1. Número de operaciones por tamaños de lotes y tiempo de operación en (ms) del merge elegido. 23
6.2. Número de operaciones por tamaños de lotes y tiempo de operación en (ms) de otros merges. 24

Capítulo 1

Introducción

En esta práctica hemos implementado una estructura basada en listas enlazadas, concretamente listas doblemente enlazadas circulares.

A partir de la clase común `DoubleLinkedList` hemos creado una variante sin orden `UnorderedDoubleLinkedList` y otra con orden `OrderedDoubleLinkedList`.

El nodo `Node<T>` guarda referencias a anterior y a siguiente, lo que nos permite recorrer la lista de manera bidireccional conectando el primero con el último.

La lista ordenada tiene la restricción de que ha de implementar elementos que implementen `Comparable<T>`, mientras que la no ordenada tiene más libertad en cuanto a los elementos que almacena.

Se han realizado pruebas tanto con enteros y con objetos `Persona` para finalmente implementarlo en la práctica 1 de Actores y películas.

1.1. Objetivos y alcance

Los objetivos principales son:

- Especificar, diseñar e implementar en Java los métodos de las clases `DoubleLinkedList` y `UnorderedDoubleLinkedList`.
- Programa de pruebas diseñado y complejidad de cada método.
- Sustituir la nueva clase `UnorderedDoubleLinkedList` en alguna de las listas usadas en la fase 1 de la práctica, y comprobar que funciona correctamente (por ejemplo, en la lista de actores que participan en una película)
- Implementación de la clase `OrderedDoubleLinkedList` es opcional

Por lo que hemos agregado comentarios a los métodos indicando su complejidad, hemos preparado unos Junits para probar las funcionalidades de `DoubleLinkedList`, `UnorderedDoubleLinkedList` y `OrderedDoubleLinkedList`. Hemos implementado las listas ordenadas en el proyecto de la práctica 1 para las listas generales de actores y películas, y hemos dejado las listas no ordenadas para las listas de películas de cada actor y las listas de actores de cada película.

Capítulo 2

Diseño de las clases

En esta sección se mencionarán las clases principales planteadas para la solución al problema. Por cada clase, se presentarán sus características, atributos y métodos, especificando por cada uno sus propiedades (visibilidad, parámetros, tipo de los resultados...). En la siguiente figura se muestran las clases y sus relaciones.

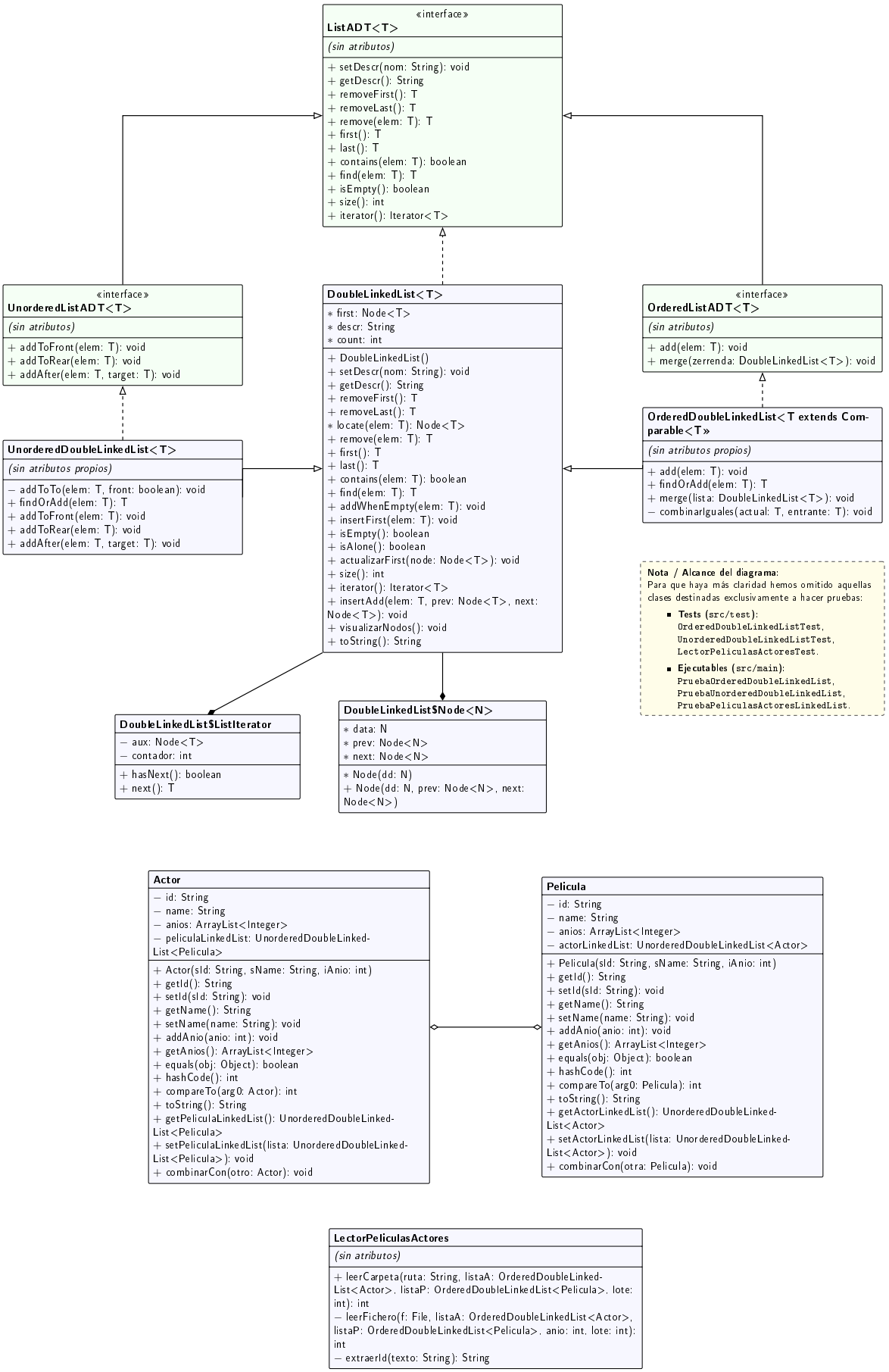


Figura 2.1: Diagrama de clases.

Capítulo 3

Descripción de las estructuras de datos principales

Sobre las listas doblemente enlazadas, podemos afirmar que si la lista está vacía, no hay ningún nodo, por lo que `first == null` y el contador vale cero `count == 0`. En caso de tener elementos cada nodo desde el primero apuntará tanto al siguiente como al anterior, de manera que el último es accesible desde el primero y el primero desde el último formando un círculo. En los casos en los que solo hay un nodo, este se apuntará a sí mismo. El contador nos permite saber el número de elementos de manera directa `O(1)` sin tener que recorrer toda la lista contando elementos, a coste de que cada vez que añadimos o quitamos un elemento de la lista agregar una modificación al contador. Cada vez que se añade o se retira un elemento de la lista, hay que actualizar los enlaces del elemento nuevo y de los elementos que rodean al nuevo elemento para que no se rompa la lista.

Capítulo 4

Diseño e implementación de los métodos principales

A continuación se presentan los métodos principales de las listas, con su código y el coste de cada uno. En los costes, n es el número de elementos de la lista y comparar dos elementos cuesta $O(1)$. Los métodos `isEmpty`, `size`, `isAlone`, `first`, `last` y `addWhenEmpty` hacen un número fijo de operaciones $\Rightarrow O(1)$.

4.1. isEmpty

```
1  //comprueba si la lista está vacía.
2  public boolean isEmpty() {return first == null;}
```

4.2. size

```
1  //devuelve el número de elementos de la lista.
2  public int size() {return count;}
```

4.3. isAlone

```
1  //comprueba si la lista contiene un único nodo.
2  public boolean isAlone() {return first == first.next;}
```

4.4. locate

```
1  //busca el nodo cuyo dato es igual a {@code elem}.
2  protected Node<T> locate(T elem) {
3    if (isEmpty()) {return null;}
4    Node<T> aux = first;
5    do {
6      if (Objects.equals(elem, aux.data)) {return aux;}
7      aux = aux.next;
8    } while (aux != first);
9    return null;
10 }
```

Coste: mejor caso (es el primero): 1 comparación $\Rightarrow O(1)$. Peor caso (no está o es el último): n comparaciones de coste constante $\Rightarrow O(n)$.

4.5. removeFirst

```
1  //Elimina el primer nodo, actualiza {@code first} y los enlaces entre el
     nuevo primero y el último.
2  public T removeFirst() {
3    Node<T> aux;
4    if (isEmpty()) {
5      throw new NoSuchElementException();
6    } else {
7      aux = first;
8      if (isAlone()) {
9        first = null;
10     } else {
11       first.prev.next = first.next;
12       first.next.prev = first.prev;
13       first = first.next;
14     }
15     count--;
16   }
17   return aux.data;
18 }
```

Coste: sin recorridos, solo comprobaciones y un número fijo de asignaciones $\Rightarrow O(1)$.

4.6. removeLast

```
1  //Elimina el último nodo y actualiza los enlaces entre el nuevo último y {
     @code first}.
2  public T removeLast() {
3    Node<T> aux;
4    if (isEmpty()) {
5      throw new NoSuchElementException();
6    } else {
7        aux = first.prev;
8        if (isAlone()) {
9          first = null;
10       } else {
11         first.prev.prev.next = first;
12         first.prev = first.prev.prev;
13       }
14       count--;
15   }
16   return aux.data;
17 }
```

Coste: el último es `first.prev` (lista circular), así que no hay recorrido $\Rightarrow O(1)$. En una lista simple sería $O(n)$.

4.7. remove

`remove` se apoya en 4.4 `locate`

```
1   //Busca y elimina el primer elemento igual a {@code elem}, actualizando los
      enlaces de sus nodos vecinos.
2   public T remove(T elem) {
3     if (isEmpty()) {throw new NoSuchElementException();}
4
5     Node<T> aux = locate(elem);
6     if (aux == null) {return null;}
7     if (aux == first) {return removeFirst();}
8
9     aux.prev.next = aux.next;
10    aux.next.prev = aux.prev;
11    count--;
12
13    return aux.data;
14  }
```

Coste: se busca con `locate` y se enlazan los vecinos: $O(n) + O(1) = O(n)$.

4.8. first

```
1   //Devuelve el primer elemento
2   public T first() {
3     if (isEmpty()) {
4       throw new NoSuchElementException();
5     } else {
6       return first.data;
7     }
8   }
```

4.9. last

```
1   //Devuelve el último elemento
2   public T last() {
3     if (isEmpty()) {
4       throw new NoSuchElementException();
5     } else {
6       return first.prev.data;
7     }
8   }
```

4.10. contains

`contains` se apoya en 4.4 `locate`

```
1   //Comprueba si la lista contiene un elemento igual a {@code elem}
2   public boolean contains(T elem) {
3     Node<T> aux;
4     if (isEmpty()) {
5       return false;
6     } else {
7       aux = locate(elem);
8       if (aux != null) {
9         return Objects.equals(aux.data, elem);
10      } else {
11        return false;
12      }
13    }
14  }
```

Coste: solo llama a `locate` $\Rightarrow O(n)$.

4.11. find

`find` se apoya en 4.4 `locate`

```
1     public T find(T elem) {
2         Node<T> aux;
3         if (isEmpty()) {
4             return null;
5         } else {
6             aux = locate(elem);
7             if (aux == null) {
8                 return null;
9             }
10            if (Objects.equals(aux.data, elem)) {
11                return aux.data;
12            } else {
13                return null;
14            }
15        }
16    }
```

Coste: igual que `contains`, solo depende de `locate` $\Rightarrow O(n)$.

4.12. addWhenEmpty

```
1   //Crea el único nodo de la lista vacía y enlaza sus referencias {@code next}
      y {@code prev} consigo mismo.
2   public void addWhenEmpty(T elem) {
3     Node<T> newNode = new Node<>(elem);
4     newNode.next = newNode;
5     newNode.prev = newNode;
6     first = newNode;
7     count++;
8   }
```

4.13. insertAdd

`insertAdd` inserta entre dos nodos que ya recibe, por lo que no recorre la lista.

```
1  //Inserta {@code elem} entre los nodos {@code prev} y {@code next},
     actualizando {@code prev.next} y {@code next.prev}.
2  public void insertAdd(T elem, Node<T> prev, Node<T> next) {
3    Node<T> newNode = new Node<>(elem, prev, next);
4    prev.next = newNode;
5    next.prev = newNode;
6    count++;
7  }
```

Coste: crear el nodo y actualizar `prev.next`, `next.prev` y `count` $\Rightarrow O(1)$.

4.14. actualizarFirst

```
1  //Cambia la referencia al primer nodo; no modifica los enlaces ni el número
     de elementos.
2  public void actualizarFirst(Node<T> node) {first = node;}
```

Coste: una sola asignación $\Rightarrow O(1)$.

4.15. insertFirst

```
1  //Inserta {@code elem} al principio y actualiza {@code first}
2  public void insertFirst(T elem) {
3    if (isEmpty()) {
4      addWhenEmpty(elem);
5      return;
6    }
7    insertAdd(elem, first.prev, first);
8    first = first.prev;
9  }
```

Coste: `insertAdd` y mover `first`: $O(1) + O(1) = O(1)$.

4.16. iterator

```
1  public Iterator<T> iterator() {return new ListIterator();}
2  private class ListIterator implements Iterator<T> {
3    private Node<T> aux = first;
4    private int contador = count;
5    @Override
6    public boolean hasNext() {return contador > 0;}
7    @Override
8    public T next() {
9      if (!hasNext()) {throw new NoSuchElementException();}
10     T data = aux.data;
11     aux = aux.next;
12     contador--;
13     return data;
14   }
15 }
```

Coste: `hasNext` y `next` cuestan $O(1)$. Recorrer toda la lista: $n \cdot O(1) = O(n)$.

4.17. toString

```
1   //Concatena la representación de todos los elementos.
2   public String toString() {
3     String result = new String();
4     Iterator<T> it = iterator();
5     while (it.hasNext()) {
6       T elem = it.next();
7       result = result + "[" + elem.toString() + "] \n";
8     }
9     return "DoubleLinkedList " + result + "]";
10  }
```

Coste: el recorrido es $O(n)$, pero cada concatenación copia todo lo acumulado:

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} \;\Rightarrow\; O(n^2)$$

4.18. Resumen de costes comunes

Operación	complejidad
`isEmpty`	$O(1)$
`size`	$O(1)$
`isAlone`	$O(1)$
`locate`	$O(n)$
`removeFirst`	$O(1)$
`removeLast`	$O(1)$
`remove`	$O(n)$
`first`	$O(1)$
`last`	$O(1)$
`contains`	$O(n)$
`find`	$O(n)$
`addWhenEmpty`	$O(1)$
`insertAdd`	$O(1)$
`actualizarFirst`	$O(1)$
`insertFirst`	$O(1)$
`iterator`	$O(1)$
`toString`	$O(n^2)$

Tabla 4.1: Costes de las operaciones comunes.

4.19. addTo

```
1  //Inserta un elemento al principio o al final
2  private void addTo(T elem, boolean front) {
3    if (isEmpty()) {
4      addWhenEmpty(elem);
5    } else {
6      insertAdd(elem, first.prev, first);
7      if (front) {actualizarFirst(first.prev);}
8    }
9  }
```

Coste: `insertAdd` (y `actualizarFirst` si es al principio) $\Rightarrow O(1)$.

4.20. UnorderedfindOrAdd

```
1  //Busca un elemento igual a {@code elem} o lo inserta en orden. {@code n} es
     el tamaño de la lista.
2  public T findOrAdd(T elem) {
3    if (isEmpty()) {
4      addWhenEmpty(elem);
5      return elem;
6    }
7    Node<T> aux = locate(elem);
8    if (aux == null) {addToRear(elem);}
9    return elem;
10 }
```

Coste: `locate` y después `addToRear`: $O(n)+O(1) = O(n)$. Mejor caso (vacía o es el primero): $O(1)$.

4.21. addToFront

```
1  //Inserta {@code elem} entre el primero y el ultimo y actualiza {@code first}
       y los enlaces entre el nuevo nodo y los extremos de la lista.
2  public void addToFront(T elem) {
3    addTo(elem, true);
4  }
```

4.22. addToRear

```
1  //Inserta {@code elem} entre el primero y el ultimo y actualiza los enlaces
     entre el nuevo nodo y los extremos de la lista.
2  public void addToRear(T elem) {
3    addTo(elem, false);
4  }
```

Coste: `addToFront` y `addToRear` solo llaman a `addTo` $\Rightarrow O(1)$.

4.23. addAfter

```
1  //Busca {@code target} e inserta {@code elem} a continuación, actualizando
     los enlaces de los nodos vecinos.
2  //Si la lista está vacía, inserta primero {@code target} y después {@code
     elem}.
3  //Si no está vacía y no encuentra {@code target}, no modifica la lista.
4  public void addAfter(T elem, T target) {
5    if (isEmpty()) {
6      addWhenEmpty(target);
7      addAfter(elem, target);
8    } else {
9      Node<T> nodoTarget = locate(target);
10     if (nodoTarget != null) {insertAdd(elem, nodoTarget, nodoTarget.next);}
11   }
12 }
```

Coste: lista no vacía: `locate` y `insertAdd`, $O(n) + O(1) = O(n)$. Lista vacía: se inserta `target` y se encuentra a la primera $\Rightarrow O(1)$.

4.24. Resumen de costes de la lista no ordenada

Operación	complejidad
`addTo`	$O(1)$
`findOrAdd`	$O(n)$
`addToFront`	$O(1)$
`addToRear`	$O(1)$
`addAfter`	$O(n)$

Tabla 4.2: Costes de las operaciones de la lista no ordenada.

4.25. add

```
1   //Inserta {@code elem} en orden si no existe. {@code n} es el tamaño de la
      lista.
2   public void add(T elem) {
3     if (isEmpty()) {
4       addWhenEmpty(elem);
5       return;
6     }
7     Node<T> last = first.prev;
8     if (elem.compareTo(first.data) == 0 || elem.compareTo(last.data) == 0) {
      return;}
9     if (elem.compareTo(last.data) > 0) {
10      insertAdd(elem, last, first);
11      return;
12    }
13    if (elem.compareTo(first.data) < 0) {
14      insertAdd(elem, last, first);
15      actualizarFirst(first.prev);
16      return;
17    }
18    Node<T> current = first;
19    while (current.next != first && elem.compareTo(current.next.data) > 0) {
      current = current.next;}
20    if (elem.compareTo(current.next.data) == 0) {return;}
21    insertAdd(elem, current, current.next);
22  }
```

Coste: mejor caso: se compara solo con `first` y `last` $\Rightarrow O(1)$. Peor caso: el elemento queda entre medias y se recorren hasta n nodos $\Rightarrow O(n)$.

4.26. OrderedfindOrAdd

```
1  //Busca un elemento igual a {@code elem} o lo inserta en orden. {@code n} es
     el tamaño de la lista.
2  public T findOrAdd(T elem) {
3    if (isEmpty()) {
4      addWhenEmpty(elem);
5      return elem;
6    }
7    int comparacionConPrimero = elem.compareTo(first.data);
8    if (comparacionConPrimero == 0) {return first.data;}
9    if (comparacionConPrimero < 0) {
10     insertAdd(elem, first.prev, first);
11     actualizarFirst(first.prev);
12     return elem;
13   }
14   Node<T> last = first.prev;
15   int comparacionConUltimo = elem.compareTo(last.data);
16   if (comparacionConUltimo == 0) {return last.data;}
17   if (comparacionConUltimo > 0) {
18     insertAdd(elem, last, first);
19     return elem;
20   }
21   Node<T> current = first;
22   while (elem.compareTo(current.next.data) > 0) {current = current.next;}
23   int comparacionConSiguiente = elem.compareTo(current.next.data);
24   if (comparacionConSiguiente == 0) {return current.next.data;}
25   insertAdd(elem, current, current.next);
26   return elem;
27 }
```

Coste: igual que `add`: mejor caso $O(1)$, peor caso $O(n)$.

4.27. merge

```
1  //En recorridos e inserciones: fusiona ambas listas en orden. {@code n} y {
     @code m} son sus tamaños iniciales; combinar elementos iguales añade el
     coste de {@code combinarCon}.
2  public void merge(DoubleLinkedList<T> lista) {
3    if (lista == null || lista.first == null) {return;}
4    if (isEmpty()) { // si nuestra lista esta vacia se clona con la del
     parametro
5      first = lista.first;
6      count = lista.count;
7      return;
8    }
9    Node<T> p1 = first; // creamos los punteros para que recorran las listas
10   Node<T> p2 = lista.first;
11   int cuantos1 = count; // guardamos en auxiliares la cantidad total de
     elementos de cada lista.
12   int cuantos2 = lista.count;
13   while (cuantos1 > 0 && cuantos2 > 0) { // fusionamos ambas listas
14     if (p1.data.compareTo(p2.data) < 0) { // si esta ordenado, seguimos
     comprobando
15       p1 = p1.next;
16       cuantos1--;
17     } else if (p1.data.compareTo(p2.data) == 0) {
18       combinarIguales(p1.data, p2.data);
19       p1 = p1.next;
20       p2 = p2.next;
21       cuantos1--;
22       cuantos2--;
23     } else { // si no reemplazamos
24       insertAdd(p2.data, p1.prev, p1); // si esta desordenado, añadimos en
     nuestra lista el elemento anterior a p1.
25       if (p1 == first) { // comprobamos si hemos añadido un elemento antes
     del first
26         actualizarFirst(first.prev); // p2 pasa a ser el nuevo first de
     nuestra lista
27       }
28       p2 = p2.next; // actualizamos el puntero de la lista del segundo
29       cuantos2--;
30     }
31   }
32   // si aun quedan elementos en la otra lista, entonces es que quedan los
     mayores, por lo que irian al final.
33   while (cuantos2 > 0) {
34     insertAdd(p2.data, first.prev, first); // insertamos cada elemento al
     final en orden
35     p2 = p2.next;
36     cuantos2--;
37   }
38 }
```

Coste: n y m son los tamaños de las dos listas. Si alguna está vacía, $O(1)$ (con la actual vacía solo se copian `first` y `count`). En el caso general cada vuelta del bucle avanza en al menos una lista y cuesta $O(1)$:

$$\text{vueltas} \leq n + m \;\Rightarrow\; O(n+m)$$

Cada pareja de elementos iguales suma una llamada a `combinarIguales`.

4.28. combinarIguales

```
1  //Combina las X de {@code otro} con las de este Y. {@code n} es el número de
     X de {@code otro}, {@code m} el número inicial de X de este Y y {@code k}
     el máximo de Y asociados a una X.
2  //Siendo X Actor o Pelicula e Y el que no sea X
3  private void combinarIguales(T actual, T entrante) {
4    if (actual instanceof Actor actor) {
5      actor.combinarCon((Actor) entrante);
6    } else if (actual instanceof Pelicula pelicula) {
7      pelicula.combinarCon((Pelicula) entrante);
8    }
9  }
```

Coste: solo llama a `combinarCon` (de `Actor` o de `Pelicula`), así que cuesta lo mismo que ese método. Para dos actores, n son las películas del actor que entra, m las que ya tenía el nuestro y k el máximo de actores de una película (no son las letras de `merge`). Por cada una de las n películas hay dos `findOrAdd`, uno en las películas del actor ($\leq m+n$ elementos) y otro en los actores de la película ($\leq k$):

$$n \cdot (O(m+n) + O(k)) = O(n(m+n+k))$$

Con dos películas es igual, intercambiando los papeles.

4.29. Resumen de costes de la lista ordenada

Operación	complejidad
`add`	$O(n)$
`OrderedfindOrAdd`	$O(n)$
`merge`	$O(n+m)$ + llamadas a `combinarIguales`
`combinarIguales`	$O(n(m+n+k))$

Tabla 4.3: Costes de las operaciones de la lista ordenada.

4.30. Pruebas realizadas

Se han implementado 18 pruebas con JUnit 5, agrupadas por clase.

Lista ordenada (`OrderedDoubleLinkedListTest`)

- `testOrdenYDuplicados`: inserta C, A, B y un A repetido; comprueba que quedan 3 elementos en orden A, B, C.
- `testFindOrAdd`: `findOrAdd` con un elemento nuevo y con uno repetido; el tamaño no crece con el repetido.

- `testMergeVaciaConVacia`: fusionar dos listas vacías deja la lista vacía.
- `testMergeVaciaConDatos`: fusionar una lista con datos en una lista vacía.
- `testMergeConDatosConVacia`: fusionar una lista vacía en una con datos no la modifica.
- `testMergeRangosSeparados`: listas con rangos que no se solapan (A, B con C, D).
- `testMergeRangosIntercalados`: listas con elementos intercalados (A, C con B, D); el resultado queda ordenado.
- `testMergeConIDsRepetidos`: dos `Actor` con el mismo identificador se combinan en uno que conserva los dos años.
- `testAutoFusion`: fusionar una lista consigo misma no lanza excepción.

Lista no ordenada (`UnorderedDoubleLinkedListTest`)

- `testAddAfterSinTarget`: `addAfter` con un objetivo ausente en una lista no vacía no modifica la lista.
- `testAddToRearaddAfter`: con películas, `addToRear` y `addAfter` dejan los elementos en el orden esperado.
- `testInsertarFrontRear`: `addToFront` y `addToRear` en lista vacía, de un elemento y de varios; se comprueban `first`, `last` y `size`.
- `testAddAfterListaVacia`: `addAfter` en lista vacía, con objetivo presente y con objetivo ausente.
- `testContainsYFind`: `contains` y `find` en lista vacía, con elemento presente y con elemento ausente.
- `testEliminaciones`: excepciones en lista vacía; eliminación de un elemento intermedio, del primero, del último y del único restante.

Integración con la práctica 1 (`LectorPeliculasActoresTest`)

- `leeActoresYPeliculasDelFichero`: cantidades de registros, actores y películas, algunos actores concretos y búsqueda por identificador.
- `testRelacionesActorPeliculaConLotes`: las relaciones entre actores y películas existen en ambos sentidos y no se repiten identificadores.
- `noDuplicaRelacionesActorPeliculaRepetidas`: las 11 filas de *Super López* corresponden a 10 actores únicos, porque Dani Rovira aparece en 2015 y 2018.

Capítulo 5

Código

Para una mejor lectura del código, se facilita el enlace al GitHub público:

- `https://github.com/erasmosee/EDA_Practica2_LinkedList`

Capítulo 6

Conclusiones

La práctica nos ha servido para ver el potencial que tienen las listas enlazadas cuando cumplen los criterios de estar doblemente enlazadas y de ser una lista circular, frente a listas enlazadas más simples, si bien es cierto que la memoria usada por las listas doblemente enlazadas es bastante superior a las listas enlazadas de una sola dirección, consideramos que es un coste asumible a la hora de proceder a trabajar con dichas listas.

6.1. Dificultades

Durante el desarrollo de la práctica 2, hemos ido probando distintas formas de integrarla con la práctica 1, pues era un objetivo que ya teníamos en mente desde el principio.

En un primer momento antes de implementar el método `merge`, procedíamos a volcar la información leída de los ficheros a un hashmap, ordenarlo y posteriormente lo volcábamos a una `OrderedDoubleLinkedList`, como es de suponer, la velocidad de insertar elementos ordenados en una lista ordenada es O(1) siempre que compares previamente el elemento a insertar con el first y con el prev.

Más tarde, tuvimos un método merge no cuadrático, y fue cuando empezamos a tener pequeñas listas sacadas una de cada fichero de año, las cuales obteníamos aun mediante hashmap, pero que las volcábamos a OrderedDoubleLinkedList para luego proceder a mergearlas en una única lista OrderedDoubleLinkedList, en este momento fue cuando el programa pasó de tardar de 12 minutos a 70 segundos en procesar la lista completa de actores y películas.

Una vez llegados a este punto en el que el programa tardaba "solo" 70 segundos, y lo comentábamos contentos pues sabíamos de donde veníamos, empezamos una pequeña competición por hacer un merge decente.

Mientras unos estaban con el merge, otros estaban dilucidando como reducir esos 70 segundos con los que en ese momento estábamos contentos, por lo que empezamos a pensar en hacer merge de listas más pequeñas a medida que leíamos de cada fichero.

En un principio pensamos que cada 1000 elementos podíamos hacer un merge por lo que modificamos el lector para que cuando llevase X elementos mergease lo leído a una `OrderedDoubleLinkedList` de los datos de ese año, para una vez finalizado el fichero de año, mergearlo con la `OrderedDoubleLinkedList` global.

Ya que teníamos un número, decidimos empezar a variarlo y comprobar como respondía el código, para nuestra sorpresa, el número bueno no era mil elementos, sino 128, y es en ese punto donde descubrimos que se realizaban el menor número de operaciones `compareTo` en nuestro código.

Ni que decir que cuanto más aproximábamos el número de lotes a 128 más rápido se procesaba el código, las últimas pruebas ya fueron para ajustar milésimas, es cuando habíamos llegado a menos de 3 segundos.

6.2. Tiempos y comparaciones del merge elegido

Dado que mergePruebas realizaba siempre más operaciones se optó por descartarlo y usar mergeEficaz, el cual fue renombrado a merge.

Tipo de lectura	Tipo de Merge	Tamaño lote	Tiempo (ms)	Comparaciones
LinkedList	eficaz	0100	3008	84505092
LinkedList	eficaz	0101	2972	84382285
LinkedList	eficaz	0102	2990	84233346
LinkedList	eficaz	0103	3005	84050349
LinkedList	eficaz	0104	2960	83982697
LinkedList	eficaz	0105	2993	83796540
LinkedList	eficaz	0106	2998	83708411
LinkedList	eficaz	0107	2958	83577457
LinkedList	eficaz	0108	3004	83476276
LinkedList	eficaz	0109	2907	83377290
LinkedList	eficaz	0110	3019	83296420
LinkedList	eficaz	0111	3028	83179341
LinkedList	eficaz	0112	3014	83155664
LinkedList	eficaz	0113	3033	83048387
LinkedList	eficaz	0114	2918	82956969
LinkedList	eficaz	0115	3001	82927237
LinkedList	eficaz	0116	2975	82840151
LinkedList	eficaz	0117	3174	82862973
LinkedList	eficaz	0118	3061	82731382
LinkedList	eficaz	0119	2852	82737519
LinkedList	eficaz	0120	2878	82697094
LinkedList	eficaz	0121	2975	82718085
LinkedList	eficaz	0122	2890	82650588
LinkedList	eficaz	0123	2966	82635178
LinkedList	eficaz	0124	2977	82638738
LinkedList	eficaz	0125	2918	82654824
LinkedList	eficaz	0126	2994	82634253
LinkedList	eficaz	0127	2940	82600991
LinkedList	eficaz	0128	2900	82594718
LinkedList	eficaz	0129	2972	82656716
LinkedList	eficaz	0130	2968	82667334
LinkedList	eficaz	0131	3009	82604169
LinkedList	eficaz	0132	3019	82652078
LinkedList	eficaz	0133	2939	82702540
LinkedList	eficaz	0134	3000	82766518
LinkedList	eficaz	0135	2943	82778629
LinkedList	eficaz	0136	2950	82816234
LinkedList	eficaz	0137	2939	82837771
LinkedList	eficaz	0138	2874	82834454
LinkedList	eficaz	0139	2910	82945814
LinkedList	eficaz	0140	2925	82985020

Tabla 6.1: Número de operaciones por tamaños de lotes y tiempo de operación en (ms) del merge elegido.

6.3. Tiempos y comparaciones de otros merges

Dado que mergePruebas realizaba siempre más operaciones se optó por descartarlo y usar mergeEficaz, el cual fue renombrado a merge.

Tipo de lectura	Tipo de Merge	Tamaño lote	Tiempo (ms)	Comparaciones
`HashMap`	eficaz	0000	4211	30.948.898
`HashMap`	eficaz	0000	4249	30.948.898
`HashMap`	eficaz	0000	4268	30.948.898
`HashMap`	eficaz	0000	4275	30.948.898
`HashMap`	pruebas	0000	4121	42.431.698
`HashMap`	pruebas	0000	4186	42.431.698
`LinkedList`	pruebas	0000	75663	2.359.061.629
`LinkedList`	pruebas	0000	78484	2.359.061.629
`LinkedList`	pruebas	0100	6804	198.922.278
`LinkedList`	pruebas	0250	5600	160.686.346
`LinkedList`	pruebas	0500	6148	199.115.873
`LinkedList`	pruebas	1000	8053	304.985.577
`LinkedList`	pruebas	2000	12932	505.847.776

Tabla 6.2: Número de operaciones por tamaños de lotes y tiempo de operación en (ms) de otros merges.